

Relazione Telemetria

TELEMETRIA - Introduzione, ruolo e architettura generale nel sistema Microbot

Il modulo di telemetria rappresenta uno degli elementi più cruciali all'interno dell'intero ecosistema MicroBot, perchè costituisce il punto di intersezione tra percezione, controllo e osservazione del sistema. In un'architettura complessa e distribuita come quella di Microbot, in cui più unità autonome cooperano tra loro attraverso interazioni locali e comunicazioni wireless, la possibilità di monitorare in tempo reale lo stato del sistema non è un semplice supporto visivo ma diventa una componente strutturale del progetto stesso. La telemetria non è quindi concepita come un pannello di debug secondario, bensì come una vera e propria interfaccia operativa che consente di comprendere, analizzare e controllare il comportamento emergente dello swarm robotico.

All'interno del progetto Microbot, la telemetria assume un ruolo ancora più rilevante in quanto il sistema non si basa su un controllo centralizzato rigido, ma su una combinazione di regole locali, sincronizzazione temporale e interazione fisica tramite forze magnetiche. Questo implica che il comportamento globale non è sempre deterministico in senso classico, ma emerge dall'interazione tra molteplici fattori: stato dei singoli moduli, latenza di comunicazione, condizioni fisiche di contatto, livello energetico e input dell'utente. In questo contesto, la telemetria diventa lo strumento fondamentale per rendere visibile ciò che altrimenti rimarrebbe nascosto, ovvero lo stato interno del sistema e le dinamiche che regolano l'evoluzione dello swarm.

Il modulo di telemetria sviluppato in questo progetto si presenta come una web application single-page, progettata per essere eseguita direttamente all'interno di un browser, e costruita utilizzando una combinazione di HTML per la struttura, CSS per la definizione dello stile visivo e JavaScript per la logica applicativa e l'interattività. Questa scelta architettonica non è casuale, ma risponde a una precisa esigenza di accessibilità, modularità e integrazione. Utilizzando tecnologie web standard, il sistema può essere eseguito su qualsiasi dispositivo dotato di browser moderno, inclusi computer, tablet e dispositivi mobili, senza la necessità di installazione complesse o ambienti di runtime specifici. Inoltre, la natura modulare del codice consente di integrare facilmente il modulo di telemetria con altri componenti del sistema Microbot, come il controller basato su ESP32 o l'interfaccia VR sviluppata in Unity.

Dal punto di vista funzionale, il modulo di telemetria non si limita a visualizzare dati numerici, ma costruisce una vera e propria rappresentazione visiva e dinamica del sistema. L'interfaccia è organizzata in più sezioni interconnesse, ciascuna delle quali rappresenta un livello specifico dell'architettura MicroBot. La sezione di controllo, ad esempio, integra un sistema di acquisizione video in tempo reale tramite webcam e un motore di analisi delle gesture della mano, permettendo all'utente di interagire con il sistema attraverso movimenti naturali. Questa componente introduce un primo livello di percezione, in cui il sistema traduce input visivi in segnali interpretabili, creando un ponte tra il mondo fisico e quello digitale.

Accanto a questa componente percettiva, il modulo include un motore grafico denominato Neural Motion Engine, che rappresenta una rete neurale visuale dinamica. Questa rete non ha solo una funzione estetica, ma serve a rendere visibile lo stato interno del sistema di elaborazione, mostrando in tempo reale parametri come il flusso di informazione, la densità delle connessioni e la dinamica delle attivazioni. In questo modo l'utente non si limita a vedere un risultato finale, ma può osservare il processo che porta a tale risultato, ottenendo una comprensione più profonda del comportamento del sistema.

La sezione di telemetria vera e propria è suddivisa in due componenti principali: la telemetria dell'intelligenza artificiale e la telemetria del sistema MicroBot. La prima raccoglie e visualizza parametri legati all'analisi delle gesture e allo stato del motore neurale, come il livello di confidenza del riconoscimento, la fluidità del movimento e la frequenza di aggiornamento del tracking. La seconda si concentra invece sullo stato dello swarm robotico, includendo informazioni come il numero di nodi attivi, la densità dei collegamenti, il livello di sincronizzazione, la modalità di pattern attuale e lo stato energetico del sistema. Questa separazione riflette la struttura a livelli del progetto MicroBot, distinguendo chiaramente tra il livello di percezione e controllo e il livello di attuazione fisica.

Un ulteriore elemento fondamentale del modulo è rappresentato dal terminale operativo, che simula una console di sistema in grado di mostrare in tempo reale eventi, stati e messaggi generati dal sistema. Questa componente svolge una duplice funzione. Da un lato, fornisce un canale diretto per il debug e l'analisi tecnica, permettendo di osservare il comportamento interno del sistema in modo dettagliato. Dall'altro lato, contribuisce alla costruzione di un'identità visiva e narrativa coerente, rafforzando la percezione del sistema come piattaforma avanzata e dinamica.

Dal punto di vista progettuale, il modulo di telemetria è stato sviluppato seguendo i principi fondamentali che caratterizzano l'intero progetto MicroBot, ovvero incrementalità, separazione delle responsabilità e integrazione tra simulazione e realtà fisica. Ogni componente è stato progettato come un modulo indipendente ma interoperabile, in modo da poter essere testato, modificato e migliorato senza compromettere il funzionamento complessivo del sistema. Allo stesso tempo, l'interfaccia è stata costruita con un forte orientamento alla visualizzazione, trasformando dati astratti in elementi grafici intuitivi e dinamici.

In conclusione, il modulo di telemetria non deve essere interpretato come un semplice strumento ma come una parte integrante dell'architettura MicroBot. Esso svolge il ruolo di interfaccia tra l'utente e il sistema, tra il livello logico e quello fisico, e tra la simulazione e l'implementazione reale. Attraverso la combinazione di acquisizione visiva, elaborazione in tempo reale, rappresentazione grafica e monitoraggio continuo, la telemetria rende il sistema osservabile comprensibile e controllabile, trasformando un insieme di componenti distribuiti in un sistema coerente e interpretabile.

Telemetria – Architettura della cartella e analisi approfondita dei componenti software

L'architettura software del modulo di telemetria rappresenta uno degli aspetti più significativi dell'intero ecosistema MicroBot, non soltanto perché questo modulo costituisce la superficie visibile con cui l'utente entra in contatto, ma soprattutto perché esso traduce in una forma osservabile e interattiva una quantità molto elevata di processi interni, relazioni logiche e stati dinamici del sistema. Quando si analizza la cartella dedicata alla telemetria, si potrebbe inizialmente essere portati a pensare di trovarsi di fronte a una struttura relativamente semplice, composta da pochi file e quindi apparentemente limitata nella propria profondità tecnica. In realtà, questa impressione è solo superficiale. La presenza di un numero contenuto di componenti non indica una bassa complessità, ma piuttosto una precisa scelta progettuale orientata alla concentrazione funzionale, alla modularità e alla chiarezza delle responsabilità. In altre parole, la complessità non è dispersa in una molteplicità di file

frammentari, ma è organizzata in un piccolo insieme di elementi ad alta densità logica.

La cartella telemetria è strutturata attorno a quattro file principali: `index.html`, `style.css`, `script.js` e `settings.json`. Questa organizzazione riprende uno schema classico del web development, in cui la struttura, la presentazione e il comportamento vengono mantenuti distinti, ma nel contesto di MicroBot questa separazione assume un valore ulteriore, quasi architeturale. Non si tratta semplicemente di seguire una convenzione ordinata, ma di costruire un sistema in cui ogni livello svolga una funzione precisa e sia in grado di evolversi senza generare dipendenze inutilmente rigide. Il file HTML definisce la struttura semantica e gerarchica dell'interfaccia; il CSS costruisce l'identità visiva e il linguaggio percettivo del sistema; il JavaScript realizza la dinamica, l'interattività, la simulazione e il collegamento tra input, stati interni e output visivi; il file JSON, infine, svolge una funzione di supporto configurativo, separando i parametri dal corpo principale del codice. Questa impostazione riflette pienamente la filosofia generale di MicroBot, che privilegia la separazione dei livelli, la leggibilità del progetto e l'estensibilità futura.

La logica della cartella come micro-architettura autonoma

Per comprendere davvero il valore della cartella telemetria, bisogna considerarla non come una semplice raccolta di risorse per una pagina web, ma come una micro-architettura software autonoma, ovvero come un sottosistema con una propria coerenza interna, un proprio ciclo di vita e una propria funzione sistemica all'interno del progetto più ampio. Essa riceve input dall'utente, li interpreta, aggiorna rappresentazioni visive, produce uno stato osservabile e, soprattutto, costruisce un ponte tra dimensione tecnica e dimensione comunicativa. Il modulo di telemetria non serve infatti solo a far funzionare qualcosa, ma a rendere interpretabile ciò che sta funzionando. Questa caratteristica è fondamentale, perché in un progetto come MicroBot l'osservabilità del sistema ha quasi lo stesso valore del sistema stesso. Un sistema complesso che non può essere letto, analizzato e presentato in modo chiaro resta opaco; un sistema complesso che invece è accompagnato da un buon modulo di telemetria diventa comprensibile, documentabile e controllabile.

Dal punto di vista progettuale, la cartella telemetria si comporta quindi come una forma di "nucleo interattivo" dell'ecosistema. Essa integra elementi di dashboard, elementi di simulazione,

elementi di interfaccia uomo-macchina, componenti di monitoring visivo e persino una dimensione narrativa, dovuta alla presenza del terminale, delle animazioni, del preloader e dell'estetica complessiva dell'interfaccia. Non siamo dunque davanti a una pagina informativa, ma a una struttura ibrida che mescola rappresentazione, controllo, estetica, percezione e feedback operativo.

Il file `index.html` come struttura semantica e gerarchica del sistema

Il file `index.html` rappresenta il fondamento statico dell'intera architettura della cartella. Il suo compito non è eseguire logica, né definire la forma estetica finale, ma predisporre il telaio strutturale su cui gli altri due livelli principali – CSS e JavaScript – possono agire. In termini architetturali, `index.html` può essere interpretato come lo scheletro del sistema di telemetria, ovvero come la mappa semantica delle sue sezioni, dei suoi pannelli, dei suoi contenitori logici e dei punti di appoggio su cui si innesteranno comportamenti dinamici e trasformazioni visive.

Un aspetto molto importante del file HTML è che non si limita a dichiarare genericamente dei blocchi, ma organizza la pagina in sezioni chiaramente distinte, ciascuna associata a una specifica funzione all'interno dell'esperienza utente. Dalla struttura emerge una suddivisione che include aree come la sezione introduttiva, la sezione di controllo, la sezione di telemetria, il terminale e ulteriori blocchi informativi. Questa progressione non è soltanto estetica o editoriale, ma segue una logica di flusso: prima l'utente viene introdotto al sistema, poi entra nella zona di interazione, poi osserva lo stato e i dati, poi accede a una rappresentazione più tecnica e operativa. Questa gerarchia produce un'esperienza in cui la pagina accompagna l'utente da un livello più generale a uno più specifico, senza chiedergli di comprendere tutto immediatamente.

La sezione di controllo, in particolare, è una delle più significative dal punto di vista architetturale. Al suo interno vengono definiti i contenitori che ospitano il feed video della webcam e il motore neurale visuale. Questi due pannelli sono centrali perché esprimono simbolicamente e tecnicamente il cuore del modulo: da una parte la percezione del mondo fisico attraverso la camera, dall'altra la rappresentazione del processo interno di elaborazione e risposta. Il file HTML stabilisce quindi una corrispondenza strutturale tra input e processamento, tra visione e computazione, tra realtà e modello.

La sezione dedicata alla telemetria vera e propria organizza poi ulteriori pannelli informativi, che possono essere interpretati come dashboard specializzate: una orientata allo stato del livello AI/gesture/vision, l'altra orientata allo stato del livello MicroBot/swarm/energia/sincronizzazione. Anche qui la struttura non è casuale. Essa riproduce una divisione architettuale interna al progetto, in cui il livello di percezione e quello di attuazione fisica sono distinti ma collegati. Il fatto che questa distinzione sia visibile nella struttura HTML dimostra che il file non è stato costruito soltanto pensando alla disposizione grafica, ma tenendo conto del modello concettuale generale del sistema.

Un ulteriore elemento da sottolineare è la presenza, nel file HTML, di componenti accessorie ma estremamente importanti per la percezione complessiva della piattaforma: barra di navigazione, pulsanti di tema, menu mobile, aree dedicate a notifiche, preloader, canvas di sfondo, elementi di supporto al movimento nella pagina e terminale. Questi componenti mostrano che l'interfaccia non è pensata come un semplice layout di pannelli, ma come un vero ambiente digitale coerente, dotato di continuità e identità. La loro presenza aumenta la complessità architettuale del file HTML e, al tempo stesso, testimonia la volontà di costruire un'esperienza di sistema e non una dimostrazione frammentaria.

Il file style.css come sistema percettivo e grammatica visiva

Se `index.html` rappresenta lo scheletro semantico del modulo, il file `style.css` ne costituisce il sistema percettivo, cioè il livello attraverso cui il modulo acquisisce identità, atmosfera, leggibilità e coerenza visiva. Ridurre il CSS a semplice "grafica" sarebbe profondamente sbagliato, perché in un progetto come questo il linguaggio visivo non è decorativo, ma funzionale alla comprensione e alla trasmissione del significato del sistema. La telemetria deve apparire come una piattaforma avanzata, reattiva, tecnica, immersiva; il CSS è lo strumento attraverso cui questa percezione viene costruita.

Dal punto di vista strutturale, il file CSS appare suddiviso in macro-sezioni che corrispondono ai diversi ambiti funzionali dell'interfaccia: preloader, cursore personalizzato, barra di navigazione, menu mobile, sistema generale, hero section, control section, camera panel, neural panel, telemetry, terminal, notifiche, footer, adattamento responsive e ulteriori effetti di supporto. Questa segmentazione interna testimonia una progettazione ordinata, in cui il foglio di stile non cresce in

maniera caotica ma viene articolato per domini di responsabilità visiva.

La funzione del CSS non è solo definire colori, dimensioni o spaziature, ma anche stabilire una gerarchia di importanza visiva tra i diversi elementi. Attraverso il foglio di stile, i pannelli di controllo acquistano un'identità differente rispetto alle aree di testo, la sezione hero assume il ruolo di ingresso visivo e narrativo, la telemetria appare come dashboard tecnica e il terminale come console operativa. Questo significa che il CSS implementa una vera gerarchia informativa, in cui l'occhio dell'utente viene guidato verso i punti più rilevanti in base all'uso di contrasti, allineamenti, sfocature, profondità, glow, box model e animazioni.

È inoltre importante osservare che il linguaggio visivo adottato è coerente con il posizionamento del progetto. L'interfaccia dark, futuristica, ad alto contrasto e arricchita da effetti di luce controllata comunica immediatamente l'idea di una piattaforma avanzata, tecnica, quasi da laboratorio sperimentale o sistema di controllo. Questo aspetto non è secondario, perché la percezione del progetto da parte di un osservatore esterno dipende fortemente da come esso si presenta. Un modulo tecnicamente valido ma esteticamente povero viene spesso percepito come incompleto; al contrario, un modulo che coniuga tecnica e cura visiva suggerisce un livello di maturità progettuale molto più elevato.

Il CSS assume anche un ruolo chiave nell'usabilità, soprattutto grazie all'adattamento responsive. La possibilità che il modulo di telemetria venga eseguito su schermi diversi richiede che la disposizione degli elementi, le dimensioni dei pannelli, la leggibilità del testo e la navigazione rimangano funzionali in contesti differenti. Questo significa che il foglio di stile non opera soltanto sul piano estetico, ma partecipa direttamente all'architettura operativa del sistema, garantendo accessibilità e continuità d'uso.

Va poi sottolineato che molti degli effetti visivi presenti nel modulo – ad esempio preloader, glow, transizioni, terminal animation, hero reveal, interazioni tra hover e stato – non servono solo a “fare scena”, ma a rafforzare l'idea che il sistema sia vivo, in attività, in esecuzione continua. Questo è particolarmente importante in un modulo di telemetria, dove la sensazione di attività permanente contribuisce a rendere più credibile il comportamento del sistema monitorato.

Il file script.js come nucleo operativo del modulo

Il vero centro dinamico della cartella telemetria è però il file `script.js`. Se HTML e CSS preparano la struttura e il linguaggio visivo, è il JavaScript a trasformare il modulo in un sistema vivo, capace di reagire, simulare, aggiornarsi, leggere input, produrre output e coordinare i diversi sottosistemi interni. `script.js` non è un file accessorio, ma il cuore operativo del modulo. Al suo interno convergono funzioni di gestione dell'interfaccia, funzioni di rendering su canvas, logiche di simulazione, interpretazione di gesture, aggiornamento dati di telemetria, gestione terminale, inizializzazione e controllo della webcam, aggiornamento di stato dell'interfaccia e coerenza temporale del sistema.

La prima cosa che colpisce nella struttura di `script.js` è la presenza di numerose funzioni con responsabilità distinte, segno di un'organizzazione modulare relativamente chiara. Tra le funzioni individuabili compaiono helper generali per operazioni matematiche o di supporto, sistemi di notifica come `showToast`, gestione del preloader, controllo del cursore personalizzato, logiche di navigazione, scrolling e theme toggle, ma soprattutto componenti molto più significative dal punto di vista sistemico, come la gestione del terminale, la costruzione e animazione della rete neurale visuale, il ridimensionamento dei canvas, l'aggiornamento della telemetria della rete, il controllo della camera, l'analisi della mano, il disegno dei landmarks e l'aggiornamento della telemetria del sistema MicroBot. Questo insieme di funzioni mostra chiaramente che il file non si limita a pilotare pochi eventi DOM, ma implementa una vera e propria micro-applicazione.

Uno degli aspetti più interessanti è il fatto che il file JavaScript svolge contemporaneamente funzioni di orchestration e funzioni di simulazione. Da una parte organizza il ciclo di vita dell'interfaccia, inizializza componenti, risponde a eventi, aggiorna il DOM, sincronizza stati visivi. Dall'altra produce vere e proprie dinamiche interne, come la rete neurale visuale animata, la generazione del terminal stream e l'aggiornamento progressivo delle metriche. Questo doppio ruolo è importante, perché rende il file un punto di convergenza tra gestione dell'interfaccia e modellazione del comportamento.

La gestione del ciclo di vita dell'interfaccia

Un sottosistema fondamentale implementato in `script.js` riguarda la gestione del ciclo di vita dell'interfaccia. Il modulo di telemetria, infatti, non appare semplicemente in stato già pronto,

ma attraversa fasi diverse: caricamento iniziale, apparizione progressiva, attivazione di elementi dinamici, risposta agli input dell'utente e aggiornamento continuo. Il preloader e le funzioni collegate rappresentano il primo livello di questo ciclo, perché preparano l'ingresso dell'utente nel sistema e costruiscono la percezione di una piattaforma complessa in fase di inizializzazione.

Successivamente entrano in gioco le logiche di navigazione, di smooth scrolling, di reveal on scroll e di interazione con la navbar. Queste componenti, apparentemente periferiche, in realtà svolgono una funzione importante: garantiscono fluidità e continuità percettiva, facendo sì che il modulo venga esperito come un unico ambiente coerente piuttosto che come una successione rigida di sezioni statiche. In un progetto orientato alla presentazione di sistemi complessi, questa coerenza è molto preziosa.

Il motore della neural visual interface

Tra le parti più forti del file JavaScript c'è il motore della neural visual interface, indicato da funzioni come `resizeNNCanvas`, `buildNetwork`, `drawConnection`, `drawRecurrentLoops`, `drawNeuron`, `drawNNBackground`, `updateNNTelemetry` e `animateNN`. Queste funzioni suggeriscono la presenza di un sistema canvas-based dedicato alla costruzione e all'animazione di una rete neurale visuale. Il ruolo di questo sottosistema è duplice.

Da un lato, esso costruisce una rappresentazione simbolica dell'attività interna del sistema. La rete di nodi e connessioni animate trasmette l'idea di un processo computazionale attivo, di un'intelligenza artificiale in funzione, di un motore di decisione o elaborazione in corso. Dall'altro lato, questo motore offre un valore didattico e comunicativo: rende visibile ciò che normalmente resterebbe astratto, permettendo all'utente di vedere un flusso, una struttura, una dinamica.

Da un punto di vista tecnico, questo sottosistema implica la presenza di un modello interno della rete, di logiche di ridimensionamento in base al canvas, di algoritmi di rendering iterativo e di una sincronizzazione temporale continua. L'animazione, infatti, non può essere casuale o isolata, ma deve essere aggiornata frame dopo frame, mantenendo coerenza visiva e prestazioni adeguate. Questo significa che `script.js` non è soltanto un file di automazione del DOM, ma contiene anche una componente di programmazione grafica in tempo reale.

Il sistema di webcam, hand tracking e gesture analysis

Un altro blocco centrale dell'architettura software di telemetria è il sistema di webcam e analisi delle gesture. Le funzioni `setCameraUIOnline`, `resizeHandCanvas`, `updateGestureUI`, `distance2D`, `countFingers`, `analyzeHand`, `drawHandLandmarks`, `initHands`, `startCamera` e `stopCamera` indicano con chiarezza la presenza di una pipeline completa che parte dall'acquisizione video e arriva fino all'interpretazione semplificata dello stato della mano.

Questa pipeline è particolarmente importante perché mette il modulo di telemetria in relazione diretta con il corpo dell'utente. Il sistema non si limita a visualizzare informazioni, ma legge il movimento, lo struttura, lo trasforma in dati interni e aggiorna l'interfaccia di conseguenza. Qui si manifesta una vera logica di human-machine interaction: la mano diventa un dispositivo di input naturale, la webcam il sensore di acquisizione, il canvas l'ambiente di rappresentazione, il JavaScript il livello di interpretazione.

Dal punto di vista tecnico, la presenza di funzioni come `distance2D` e `countFingers` suggerisce che il file implementi almeno una parte dell'analisi geometrica necessaria per determinare configurazioni significative della mano. La rilevazione della distanza tra landmark, il conteggio delle dita attive o aperte, l'aggiornamento dello stato UI in base alla gesture riconosciuta e il disegno dei landmarks sull'overlay canvas mostrano un livello di integrazione notevole tra visione artificiale e interfaccia. Questo è particolarmente rilevante per MicroBot, poiché l'interazione tramite gesture è uno degli elementi che distinguono il progetto rispetto a sistemi di controllo più tradizionali.

La telemetria AI come osservabilità del livello di percezione

Le funzioni legate all'aggiornamento della telemetria neurale o AI mostrano che il modulo non si limita a disegnare una rete, ma associa a essa una serie di metriche, probabilmente legate a confidence, activity, flow, pulse, frame rate, stability o parametri analoghi. Questo significa che il sistema implementa una forma di osservabilità del livello di percezione e analisi. In altre parole, l'utente non vede soltanto che la webcam è attiva o che la rete si muove, ma riceve un insieme di segnali sintetici che descrivono la qualità o l'intensità del processo in corso.

Questa è una scelta molto intelligente, perché in un sistema complesso non basta rappresentare i processi: bisogna anche sintetizzarli in forme leggibili. Le metriche di telemetria AI agiscono quindi come indicatori di stato, rendendo il comportamento del sistema interpretabile a colpo d'occhio. Questa capacità di sintesi è tipica delle dashboard ben progettate.

La telemetria MicroBot come ponte verso il sistema fisico

Parallelamente alla telemetria del livello AI, il file JavaScript include anche una funzione esplicitamente orientata allo stato del sistema robotico, indicata come `updateMicroBotTelemetry`. La presenza di questo sottosistema è di enorme importanza, perché collega la dashboard a ciò che nel progetto rappresenta il nucleo fisico e distribuito: lo swarm dei MicroBot. Questo significa che la telemetria non resta confinata al livello di interfaccia e percezione, ma si spinge fino a rappresentare il comportamento di un insieme di nodi, i loro parametri di attività, la loro sincronizzazione, la loro energia o la densità delle connessioni.

Dal punto di vista sistemico, questa funzione svolge il ruolo di ponte tra due mondi: il mondo della simulazione e dell'interazione, e il mondo della robotica modulare vera e propria. Anche quando i dati mostrati sono simulati o parzialmente simbolici, l'interfaccia prepara già il terreno per un collegamento futuro con dati reali provenienti dal controller o dai bot. Questo rende la dashboard non solo una demo visiva, ma una base concreta per un sistema di monitoraggio reale. Ed è proprio questo uno dei punti forti dell'intero progetto MicroBot: la costante tendenza a costruire componenti che siano già pensati per essere estesi verso l'hardware effettivo.

Il terminale come interfaccia tecnica e dispositivo narrativo

Un elemento che merita un'analisi a parte è il terminale operativo. La presenza di una funzione come `pushTerminalLine` indica che il modulo genera uno stream di messaggi che vengono aggiunti progressivamente alla console interna. Questo componente svolge una funzione tecnica evidente: mostra eventi, log, aggiornamenti di stato e segnali di attività. Tuttavia, la sua importanza va oltre la semplice utilità pratica.

Il terminale costruisce infatti una specifica atmosfera percettiva: quella di un sistema in esecuzione continua,

osservabile, "vivo". Esso rafforza l'identità del progetto come piattaforma tecnica avanzata e allo stesso tempo agisce come meccanismo di narrazione interna. Attraverso le linee che scorrono, il modulo comunica all'utente che esiste un livello di profondità ulteriore, una dimensione operativa sottostante che produce continuamente eventi. In questo senso, il terminale è sia uno strumento di monitoring sia un dispositivo di storytelling tecnico.

Il ruolo di settings.json nella configurazione del sistema

Sebbene meno appariscente rispetto agli altri tre file principali, **settings.json** riveste un ruolo importante nella logica generale della cartella. La separazione dei parametri configurabili dal corpo principale del codice è infatti una pratica molto utile in qualsiasi sistema che si voglia mantenere modulare ed estensibile. Questo approccio consente di modificare determinati aspetti del comportamento del modulo senza dover intervenire direttamente sul file JavaScript, riducendo il rischio di introdurre errori e facilitando l'adattamento del sistema a nuove esigenze.

Anche qualora il contenuto attuale del file sia limitato, la sua semplice presenza indica una direzione progettuale chiara: il modulo di telemetria non è pensato come una pagina rigida, ma come una struttura parametrizzabile, in cui determinati valori possano essere centralizzati e aggiornati in modo controllato. Questa scelta è coerente con l'idea di far evolvere il modulo da semplice interfaccia dimostrativa a vero componente configurabile di un ecosistema più grande.

Una cartella piccola solo in apparenza

Tirando le fila dell'analisi, emerge con chiarezza che la cartella telemetria è piccola solo in apparenza. Dal punto di vista del numero di file è contenuta, ma dal punto di vista architettuale è estremamente densa. Al suo interno convivono una struttura HTML gerarchica e semantica, un sistema CSS evoluto e identitario, un motore JavaScript che integra interfaccia, simulazione, webcam, gesture, neural visualization, terminale e telemetria robotica, e infine una componente di configurazione separata. Tutto questo produce un modulo che non può essere considerato una semplice

pagina web, ma una vera interfaccia operativa per il sistema MicroBot.

La forza di questa architettura sta nel fatto che ogni livello svolge una funzione precisa e leggibile. Il file HTML organizza, il CSS comunica e ordina percettivamente, il JavaScript anima, collega, simula e aggiorna, il JSON parametrizza. Questa chiarezza di ruoli costituisce una base eccellente per l'evoluzione futura, perché permette di estendere il sistema senza compromettere la comprensione globale del progetto. Ed è proprio qui che si vede la maturità del modulo: non tanto nel numero di effetti o nel livello di "scena", ma nella coerenza con cui struttura, presentazione, logica e configurazione collaborano tra loro.

In conclusione, l'architettura della cartella telemetria rappresenta un esempio particolarmente significativo del metodo di sviluppo adottato in MicroBot. Essa mostra come un numero ridotto di componenti, se progettati con attenzione, possa dare origine a un sistema interattivo complesso, osservabile, estendibile e coerente. La cartella non è soltanto un contenitore di file, ma un sottosistema completo che rende visibile il comportamento di altri sottosistemi, collegando l'utente all'ecosistema MicroBot attraverso una forma di telemetria che è al tempo stesso tecnica, visiva e narrativa.

2.1 Analisi approfondita del file `index.html`

Il file `index.html` rappresenta il punto di ingresso strutturale dell'intero modulo di telemetria del sistema MicroBot. Esso non deve essere interpretato come un semplice documento HTML destinato alla visualizzazione statica di contenuti, ma come la base semantica e architetturale su cui si innestano tutti i livelli successivi del sistema, inclusi il comportamento dinamico implementato in JavaScript e il sistema visivo definito tramite CSS. In questo senso, il file svolge un ruolo fondamentale nella costruzione della coerenza complessiva del modulo, poiché definisce non solo cosa viene mostrato, ma soprattutto come le diverse componenti sono organizzate, correlate e rese disponibili all'elaborazione dinamica.

Uno degli aspetti più rilevanti del file `index.html` è la sua funzione di "mappa logica" dell'interfaccia. Ogni elemento presente nel documento non è inserito in maniera casuale, ma risponde a una precisa esigenza architetturale. I contenitori principali, le sezioni, i pannelli e i componenti secondari sono

disposti secondo una gerarchia che riflette direttamente la struttura concettuale del sistema di telemetria. Questo significa che l'HTML non è solo una rappresentazione visiva, ma una traduzione diretta della logica del sistema in una forma navigabile e organizzata.

2.1.1 Struttura generale del documento

Dal punto di vista formale, il file segue la struttura standard dei documenti HTML5, con dichiarazione del doctype, apertura del tag `<html>` e suddivisione in `<head>` e `<body>`. Tuttavia, ciò che distingue questo file da una pagina web tradizionale è il modo in cui il corpo del documento viene organizzato in macro-sezioni funzionali. Non si tratta di una semplice sequenza di blocchi, ma di una struttura stratificata in cui ogni sezione rappresenta un livello dell'interazione tra utente e sistema.

Il documento include una serie di elementi globali che preparano il contesto dell'interfaccia, tra cui meta tag per la gestione della viewport e della compatibilità, collegamenti ai fogli di stile e agli script, e definizioni che permettono una corretta visualizzazione su dispositivi diversi. Questi elementi, sebbene standard, assumono un ruolo importante perché garantiscono che il modulo di telemetria sia accessibile, responsivo e stabile in ambienti differenti.

All'interno del `<body>`, la struttura si articola in una sequenza di componenti che possono essere suddivisi in tre categorie principali: componenti di sistema, componenti di interfaccia principale e componenti di supporto. I componenti di sistema includono elementi come il preloader, il cursore personalizzato e il canvas di sfondo, che contribuiscono alla costruzione dell'ambiente visivo generale. I componenti di interfaccia principale comprendono invece le sezioni operative, come la hero section, la control section e la telemetry section. Infine, i componenti di supporto includono elementi come il terminale, le notifiche e il footer, che completano l'esperienza e aggiungono funzionalità accessorie ma rilevanti.

2.1.2 Organizzazione a sezioni e flusso dell'interfaccia

Uno degli aspetti più interessanti del file `index.html` è l'organizzazione dell'interfaccia in sezioni distinte, ciascuna con un ruolo specifico all'interno del flusso di interazione.

Questa suddivisione non è solo estetica, ma riflette un percorso logico che l'utente segue durante l'utilizzo del sistema.

La prima sezione, comunemente definita come "hero", rappresenta il punto di ingresso visivo e concettuale. Essa ha il compito di introdurre il sistema, fornire un'identità chiara e catturare l'attenzione dell'utente. In questa fase, l'utente non interagisce ancora in modo tecnico con il sistema, ma viene immerso nel contesto generale del progetto. La presenza di titoli, sottotitoli e call to action crea una prima connessione tra utente e piattaforma.

Successivamente, l'utente entra nella control section, che rappresenta il cuore operativo dell'interfaccia. In questa sezione sono presenti i pannelli principali dedicati alla webcam e alla rete neurale visuale. La disposizione di questi elementi non è casuale: la webcam rappresenta l'input del sistema, mentre la rete neurale rappresenta il processo interno di elaborazione. Questa dualità è riflessa nella struttura HTML, che separa chiaramente i due pannelli ma li mantiene all'interno della stessa area, suggerendo una relazione diretta tra acquisizione e processamento.

La sezione successiva è quella dedicata alla telemetria, in cui vengono mostrati i dati relativi allo stato del sistema. Qui l'interfaccia assume un carattere più tecnico, con pannelli informativi che riportano metriche, parametri e indicatori. La struttura HTML organizza questi elementi in contenitori che facilitano la lettura e l'aggiornamento dinamico, permettendo al JavaScript di intervenire in modo mirato su ciascun componente.

Infine, il terminale rappresenta il livello più profondo dell'interfaccia. Esso simula una console operativa, mostrando messaggi, log e informazioni in tempo reale. Dal punto di vista strutturale, il terminale è implementato come un contenitore dinamico che viene aggiornato progressivamente, creando un flusso continuo di informazioni. Questa sezione aggiunge una dimensione ulteriore all'interfaccia, trasformandola da semplice dashboard a sistema attivo e monitorabile.

2.1.3 Gerarchia semantica e accessibilità

Un elemento fondamentale nella progettazione del file `index.html` è la gerarchia semantica. L'uso corretto dei tag HTML, come `<section>`, `<div>`, `<header>`, `<nav>` e `<footer>`, permette di organizzare il contenuto in modo logico e comprensibile sia per l'utente che per il browser. Questa gerarchia non è solo una questione di ordine, ma influisce direttamente sulla leggibilità del codice, sulla manutenzione e sull'accessibilità.

La presenza di una struttura semantica ben definita consente inoltre una migliore integrazione con il CSS e il JavaScript. Gli elementi possono essere selezionati in modo più preciso, riducendo la complessità delle operazioni di manipolazione del DOM. Questo è particolarmente importante in un sistema dinamico come quello di telemetria, in cui gli aggiornamenti avvengono frequentemente e devono essere efficienti.

Dal punto di vista dell'accessibilità, la struttura semantica facilita anche l'interpretazione del contenuto da parte di tecnologie assistive. Sebbene questo aspetto non sia sempre immediatamente visibile, esso rappresenta un elemento importante nella costruzione di un sistema robusto e inclusivo.

2.1.4 Componenti interattivi e collegamento con JavaScript

Il file `index.html` non contiene logica esecutiva, ma prepara tutti i punti di aggancio necessari per il funzionamento del JavaScript. Ogni elemento interattivo, come pulsanti, canvas, pannelli e contenitori dinamici, è dotato di identificatori o classi che permettono al codice JavaScript di selezionarlo e modificarlo.

Ad esempio, i canvas utilizzati per la visualizzazione della rete neurale e dei landmark della mano sono definiti nel file HTML, ma il loro contenuto viene completamente gestito dal JavaScript. Allo stesso modo, i contenitori della telemetria sono predisposti per ricevere aggiornamenti dinamici, che vengono effettuati attraverso funzioni specifiche.

Questa separazione tra definizione e comportamento è uno dei punti di forza dell'architettura. Il file HTML definisce cosa esiste, mentre il JavaScript definisce cosa succede. Questa distinzione rende il sistema più modulare e più facile da estendere.

2.1.5 Elementi di supporto e costruzione dell'ambiente

Oltre alle sezioni principali, il file `index.html` include una serie di elementi di supporto che contribuiscono alla costruzione dell'ambiente complessivo. Tra questi troviamo il preloader, il cursore personalizzato, le notifiche e il canvas di sfondo. Questi elementi non sono strettamente necessari per il funzionamento del sistema, ma migliorano significativamente l'esperienza utente.

Il preloader, ad esempio, crea una fase di transizione tra il caricamento della pagina e la sua visualizzazione, dando l'impressione di un sistema in inizializzazione. Il cursore personalizzato aggiunge un livello di interazione visiva che rende l'interfaccia più dinamica. Le notifiche permettono di comunicare eventi o stati in modo immediato, mentre il canvas di sfondo contribuisce alla costruzione di un ambiente visivo coerente.

Questi elementi dimostrano che il file HTML non è progettato solo per contenere dati, ma per creare un'esperienza completa, in cui ogni componente contribuisce alla percezione del sistema.

2.1.6 Considerazioni architetturali finali

In conclusione, il file `index.html` svolge un ruolo centrale nell'architettura del modulo di telemetria. Esso definisce la struttura, organizza le informazioni, prepara i punti di interazione e costruisce la base su cui si sviluppano tutti gli altri livelli del sistema. La sua importanza non risiede nella complessità del codice, ma nella chiarezza con cui traduce la logica del sistema in una forma concreta e navigabile.

La qualità della struttura HTML influisce direttamente sulla qualità dell'intero modulo. Una struttura ben progettata facilita lo sviluppo, migliora la leggibilità, riduce gli errori e rende il sistema più scalabile. Nel contesto di MicroBot, il file `index.html` rappresenta quindi molto più di una semplice pagina web: è il primo livello di una architettura complessa, il punto di incontro tra teoria e implementazione, e la base su cui si costruisce l'intero sistema di telemetria.